PRACTICE TEST – API CONSUMPTION

Dataset: **Monsta Movies**

Test Server Base URL

https://t4e-demotestserver.onrender.com/api

Accessing the Dataset (Instructions)

Before solving the questions:

1. Generate a token using:

POST /api/public/token

Body

```
{
  "uniqueId": "YOUR_ID",
  "set": "setA"
}
```

2. Use the returned token to access the dataset:

GET /api/private/movies

Header

Authorization: Bearer <token>

3. Cache the dataset in memory and build your own API server.

Question 1 – Static Endpoint

Create the endpoint

GET /movies

Task

Return all movies from the cached dataset.

Example Response

```
{
  "totalMovies": 50,
  "movies": [...]
}
```

Question 2 – Dynamic Endpoint

Create the endpoint

GET /movies/:id

Task

Return movie details using the given movie ID.

Example Request

GET /movies/tt0114709

Example Response

```
{
  "id": "tt0114709",
  "name": "Toy Story",
  "genre": ["Animation", "Adventure", "Comedy"]
}
```

Error Response

```
{
  "message": "Movie not found"
}
```

Question 3 – Search Query Endpoint

Create the endpoint

GET /movies/search?genre=VALUE

Task

Return all movies that belong to the given genre.

Example Request

GET /movies/search?genre=Action

Example Response

```
[
  {
    "id": "tt0468569",
    "name": "The Dark Knight",
    "genre": ["Action", "Crime", "Drama"]
  }
]
```

Question 4 – Movies Count

Create the endpoint

GET /movies/count

Task

Return the total number of movies available in the dataset.

Example Response

```
{
  "totalMovies": 50
}
```

Question 5 – Genre List

Create the endpoint

GET /movies/genres

Task

Return all unique genres available in the dataset.

Example Response

```
{
  "genres": ["Drama", "Crime", "Action", "Adventure"]
}
```

Question 6 – Movies with Multiple Genres

Create the endpoint

GET /movies/multi-genre

Task

Return movies that belong to **more than one genre**.

Example Response

```
[
  {
    "id": "tt0468569",
    "name": "The Dark Knight",
    "genre": ["Action", "Crime", "Drama"]
  }
]
```

Question 7 – Genre Statistics

Create the endpoint

GET /movies/genre/count

Task

Return the number of movies available for each genre.

Example Response

```
{
  "Drama": 23,
  "Action": 12,
  "Adventure": 9
}
```

Question 8 – First Movie of Each Genre

Create the endpoint

GET /movies/genre/first

Task

Return the **first movie found for each genre**.

Example Response

```
{
  "Drama": "The Shawshank Redemption",
  "Action": "The Dark Knight",
  "Adventure": "The Lord of the Rings: The Fellowship of the Ring"
}
```

Question 9 – Most Frequent Genre

Create the endpoint

GET /movies/genre/popular

Task

Return the **most frequently occurring genre** in the dataset and the number of movies belonging to that genre.

Requirements

- Must compute the result dynamically.
- Must use **reduce or counting logic**.
- Must not modify the original dataset.

Example Response

```
{
  "genre": "Drama",
  "count": 23
}
```

Question 10 – Movies by Genre Count

Create the endpoint

GET /movies/genre/:genre/count

Task

Return the number of movies available for the specified genre.

Requirements

- Use **route parameter**.
- Genre comparison must be **case-insensitive**.
- If no movies found → return proper error message.

Example Request

GET /movies/genre/action/count

Example Response

```
{
  "genre": "Action",
  "count": 12
}
```

Example Error Response

```
{
  "message": "No movies found for this genre"
}
```

Final Practice Test Structure

Question	Endpoint	Concept
Q1	/movies	Static endpoint
Q2	/movies/:id	Dynamic route
Q3	/movies/search	Query parameter
Q4	/movies/count	Aggregation
Q5	/movies/genres	Unique values
Q6	/movies/multi-genre	Array filtering
Q7	/movies/genre/count	Reduce counting
Q8	/movies/genre/first	Object grouping
Q9	/movies/genre/popular	Reduce aggregation
Q10	/movies/genre/:genre/count	Dynamic + filtering